

Practice Questions: Covering Strings, Indexing & Slicing, String Methods, Lists, Tuples, Sets, Dictionaries, CRUD, Dictionary Methods, Nested Dictionaries

Comprehensive, graduated exercises with solutions – from beginner to advanced

1. Introduction

This document provides **100+ practice questions** covering all collection and string topics from the previous notes. Each question is designed to test both **syntax** and **problem-solving** skills. Solutions are provided with explanations.

How to use:

- Attempt each question without looking at the solution.
- Write code in a Python file or interactive environment.
- Compare your solution with the given one – there's often more than one correct way.

Topics covered:

1. Strings & Basic Indexing/Slicing
2. String Methods
3. Lists
4. Tuples
5. Sets
6. Dictionaries (creation, CRUD)
7. Dictionary Methods
8. Nested Dictionaries
9. Mixed / Comprehensive Problems

2. Strings – Basic Indexing & Slicing

Beginner

Q1: Given `s = "Python"` , extract the characters at indices 0, 3, and -1.

► Solution

Q2: Slice the string `"Data Science"` to get `"Sci"` .

► Solution

Q3: Use negative slicing to get the last three characters of `"abcdefg"` .

► Solution

Q4: Reverse the string `"hello"` using slicing.

► Solution

Intermediate

Q5: Given `s = "abcdefgh"` , extract every second character starting from index 1.

► Solution

Q6: Write a function that returns `True` if a string is a palindrome (reads same forwards and backwards), ignoring case.

► Solution

Q7: Given a string with digits and letters, extract only the digits using slicing and a loop (no regex).

► Solution

Advanced

Q8: Rotate a string left by `k` positions. For `s = "hello"` , `k=2` $\rightarrow$ `"llohe"` .

► Solution

Q9: Implement a function that mimics `str.find(sub)` without using built-in `find`. Return the first index or -1.

► Solution

3. String Methods

Beginner

Q10: Convert `" hello "` to uppercase and strip whitespace.

► Solution

Q11: Replace all spaces with underscores in `"I love Python"`.

► Solution

Q12: Split `"apple,banana,grape"` by comma and join with a pipe `|`.

► Solution

Intermediate

Q13: Count how many times the letter `'o'` appears in `"hello world"`.

► Solution

Q14: Capitalize the first letter of each word in `"python is awesome"` (like title, but manually).

► Solution

Q15: Check if a string contains only alphabetic characters and at least one uppercase letter.

► Solution

Advanced

Q16: Write a function that converts a string to `snake_case` (lowercase with underscores). Input:
`"FirstName" → "first_name"`.

► Solution

Q17: Implement a simple template engine: replace `{name}` with a value from a dict.

► Solution

4. Lists

Beginner

Q18: Create a list of numbers from 1 to 5, then append 6, insert 0 at the beginning, and remove 3.

► Solution

Q19: Find the sum and average of `[10,20,30,40,50]` without using `sum()` .

► Solution

Q20: Merge two lists `[1,2,3]` and `[4,5,6]` into `[1,2,3,4,5,6]` using `extend` .

► Solution

Intermediate

Q21: Given `[1,2,3,2,1,4,5,4]` , remove duplicates while preserving order.

► Solution

Q22: Rotate a list to the right by `k` positions. Example: `[1,2,3,4,5]` , `k=2` $\rightarrow$ `[4,5,1,2,3]` .

► Solution

Q23: Find the second largest element in a list without sorting.

► Solution

Advanced

Q24: Implement a function that takes a list and returns a new list where each element is the product of all other elements (without division).

► Solution

Q25: Given two sorted lists, merge them into one sorted list without using `sort()` .

► Solution

5. Tuples

Beginner

Q26: Create a tuple with elements `10, 20, 30` and try to change the second element (observe error).

► Solution

Q27: Convert a list `[1, 2, 3]` to a tuple and back to a list.

► Solution

Q28: Unpack the tuple `("Alice", 25, "Engineer")` into three variables and print.

► Solution

Intermediate

Q29: Return multiple values from a function using a tuple (e.g., min and max of a list).

► Solution

Q30: Given two tuples `(1, 2, 3)` and `(4, 5, 6)`, concatenate them into `(1, 2, 3, 4, 5, 6)`.

► Solution

Q31: Count the number of times `2` appears in `(1, 2, 2, 3, 2, 4)`.

► Solution

Advanced

Q32: Find the index of the first occurrence of `5` in `(10, 20, 5, 30, 5)`.

► Solution

Q33: Create a namedtuple `Point` with fields `x` and `y`, then compute distance from origin.

► Solution

6. Sets

Beginner

Q34: Create a set from `[1,2,2,3,3,3]` and print it.

► Solution

Q35: Add `4` and then remove `2` from the set `{1,2,3}` .

► Solution

Q36: Check if `5` is present in `{1,2,3,4}` without using a loop.

► Solution

Intermediate

Q37: Find the union and intersection of `{1,2,3}` and `{2,3,4}` .

► Solution

Q38: Given two lists, find the elements that are in the first but not in the second (set difference).

► Solution

Q39: Remove all duplicates from a string `"abracadabra"` while preserving first occurrence order using set.

► Solution

Advanced

Q40: Find the longest consecutive sequence in an unsorted list using a set ($O(n)$). Example `[100,4,200,1,3,2]` $\rightarrow$ `4` (sequence `1,2,3,4`).

► Solution

Q41: Write a function that returns the symmetric difference of two sets (elements in either but not both).

► Solution

7. Dictionaries – Creation & Basic CRUD

Beginner

Q42: Create a dictionary mapping three cities to their country capitals (e.g., "France":"Paris").

► Solution

Q43: Add a new key "Spain" with value "Madrid" to the dictionary above.

► Solution

Q44: Update the value for "Germany" to "Bonn" (just for practice, then revert if you like).

► Solution

Q45: Delete the key "Italy" from the dictionary.

► Solution

Intermediate

Q46: Write a function that takes a dictionary and a key, and returns the value if the key exists, else returns "Not Found" .

► Solution

Q47: Merge two dictionaries `d1 = {"a":1,"b":2}` and `d2 = {"b":3,"c":4}` into a new dictionary where values from `d2` override `d1` .

► Solution

Q48: Count the frequency of each word in a sentence using a dictionary.

► Solution

Advanced

Q49: Invert a dictionary (keys become values, values become keys) assuming all values are unique.

► Solution

Q50: Write a function that returns the key with the maximum value in a dictionary.

► Solution

8. Dictionary Methods

Beginner

Q51: Use `.keys()` to print all keys of `{"a":1,"b":2,"c":3}` .

► Solution

Q52: Use `.values()` to print all values.

► Solution

Q53: Use `.items()` to iterate over key-value pairs and print each.

► Solution

Q54: Use `.pop()` to remove key `"b"` and capture its value.

► Solution

Q55: Use `.setdefault()` to add a key `"d"` with value `4` if it doesn't exist.

► Solution

Intermediate

Q56: Write a function that merges two dictionaries and sums values for common keys (instead of overwriting).

► Solution

Q57: Using `popitem()` , remove and return the last inserted key-value pair (since Python 3.7).

► Solution

Q58: Create a dictionary from two lists: `keys = ["name","age"]` and `values = ["Alice",30]` using `zip()` and `dict()` .

► Solution

Advanced

Q59: Implement a `defaultdict` that groups words by their first letter.

► Solution

Q60: Use `Counter` to find the most common character in a string (excluding spaces).

► Solution

9. Nested Dictionaries

Beginner

Q61: Create a nested dictionary representing a person with keys "name" , "address" (which itself has "city" , "zip"), and "age" .

► Solution

Q62: Add a new inner key "street" under "address" .

► Solution

Q63: Access the "zip" value safely using .get() chaining.

► Solution

Intermediate

Q64: Write a function that recursively prints all keys and values of a nested dictionary with indentation.

► Solution

Q65: Flatten a nested dictionary into dot-separated keys. Example: {"a":{"b":1}} → {"a.b":1} .

► Solution

Q66: Deep merge two nested dictionaries (recursively combine, not overwrite).

► Solution

Advanced

Q67: Given a nested dict and a path (e.g., "a.b.c"), get the value or return None .

► Solution

Q68: Build a nested dictionary from a list of paths (["home/user/docs", "home/user/pics"]) where leaf values are None .

► Solution

10. Mixed / Comprehensive Problems

Q69: Given a list of dictionaries, find the dictionary with the maximum value for a specific key.

► Solution

Q70: Write a function that returns a list of keys that appear in all dictionaries in a list (common keys).

► Solution

Q71: Convert a list of pairs `[("a",1), ("b",2)]` into a dictionary using `dict()`.

► Solution

Q72: Given two lists `keys` and `values` of equal length, create a dictionary but skip any key that is `None`.

► Solution

Q73: Write a function that takes a dictionary and returns a new dictionary with keys and values swapped, but if multiple keys have the same value, keep them in a list.

► Solution

Q74: Remove all empty dictionaries from a nested dictionary (recursively).

► Solution

Q75: Given a sentence, return the word with the highest number of vowels. Use dictionary for counts.

► Solution

11. Performance & Edge Cases Practice

Q76: Compare the runtime of `list` vs `set` for membership testing on 10,000 elements.

► Solution (conceptual)

Q77: Write a function that removes duplicates from a list of unhashable items (e.g., dictionaries) by converting to a string representation.

► Solution

Q78: What happens when you use a list as a dictionary key? Why? Write code to demonstrate.

► Solution

12. Mini Project: Contact Manager (Practice All Topics)

Q79: Build a contact manager using a nested dictionary where each contact has name, phone, email, and tags (set). Implement:

- Add contact
- Search by name
- Search by tag
- Delete contact
- List all contacts sorted by name

► Solution skeleton

13. Summary Table of Practice Areas

Topic	Number of Questions	Key Methods/Concepts
Strings & Slicing	9	indexing, slicing, reverse, palindrome
String Methods	8	strip, replace, split, join, count, isalpha
Lists	8	append, insert, remove, extend, comprehension
Tuples	8	immutability, unpacking, count, index, namedtuple
Sets	8	add, remove, union, intersection, difference
Dict CRUD	9	assignment, get, del, merging
Dict Methods	10	keys, values, items, pop, setdefault, fromkeys, Counter
Nested Dict	8	chained access, recursion, flatten, deep merge
Mixed	11	real-world problems combining collections
Performance	3	big-O, hashing, unhashable types
Mini Project	1	apply everything

14. Key Takeaways from Practice

-  **Strings are immutable** – every method returns a new string.
-  **Slicing never raises an error** – it gracefully handles out-of-range indices.
-  **Lists are versatile** – but watch out for $O(n)$ insert/delete in the middle.
-  **Tuples are lightweight and hashable** – use them as dictionary keys.
-  **Sets are perfect for uniqueness and fast membership** – $O(1)$.
-  **Dictionaries are the Swiss Army knife** – learn all the methods.
-  **Nested dictionaries mirror real data** – master recursive traversal.
-  **Practice reveals patterns** – after enough exercises, you stop thinking about syntax and start solving problems.